Sistema de préstamos de equipos

Resumen de funcionamiento y entrega académica
14 de octubre de 2026

Objetivo

Administrar equipos de laboratorio, identificar al estudiante que recibe cada equipo y conservar el historial de préstamos y devoluciones. El módulo adicional de reservas permite solicitar un horario y recibir aprobación del encargado.

Cómo se conecta

Login -> sesión -> rol -> panel -> operación -> base de datos.
Laravel autentica; el middleware protege las rutas; Blade y Livewire muestran las pantallas. OperacionEquipos reúne las reglas compartidas de préstamo, devolución y reserva.

Entidad	Datos y relaciones
users	Nombre, correo, contraseña cifrada y rol (encargado o estudiante). Tiene préstamos y reservas.
equipos	Código único, nombre, categoría y estado. Tiene préstamos y reservas.
prestamos	Equipo, estudiante, fecha de préstamo, fecha de devolución y observaciones.
reservas	Equipo, estudiante, inicio, fin, estado y observaciones.

Flujos principales

Préstamo: encargado selecciona equipo disponible y estudiante; guarda el préstamo y el equipo pasa a Prestado.
Devolución: se guarda la fecha sin borrar el préstamo; el equipo vuelve a Disponible o a Reservado si conserva una reserva aprobada.
Reserva: solicitud Pendiente -> Aprobada/Rechazada. La entrega dentro del horario genera un préstamo y la reserva pasa a Finalizada.

Datos de demostración

Un encargado, dos estudiantes y ocho equipos (EQ-001 a EQ-008). Los seeders se pueden repetir sin duplicar registros ni restablecer equipos ya prestados.

Instalación y validación

Guía breve para levantar el sistema y revisar la entrega.

Requisitos y arranque

PHP 8.3 o superior, Composer 2 y Node.js 22.12 o superior. SQLite simplifica la demostración local; el README incluye la configuración de MySQL.

```
git clone https://github.com/Jamesless-dev/sistema-prestamos.git
cd sistema-prestamos
composer run setup
php artisan serve
```

Abrir **http://localhost:8000**. Para reiniciar exclusivamente una base de demostración: **php artisan migrate:fresh --seed**. Este comando elimina sus tablas.

Cuenta	Correo	Contraseña
Encargado	encargado@tecsup.edu.pe	Admin123
Estudiante 1	estudiante1@tecsup.edu.pe	Estudiante123
Estudiante 2	estudiante2@tecsup.edu.pe	Estudiante123

Reglas que protege el sistema

1. El estudiante no accede al CRUD ni gestiona préstamos.
2. No se registra un segundo préstamo activo para un equipo.
3. Una devolución no se puede registrar dos veces.
4. No se aprueban reservas que se superponen.
5. El estudiante solo cancela sus propias reservas.
6. No se borran cuentas o equipos con historial.
7. Los formularios tienen validación y protección CSRF.

Verificación de la entrega

103 pruebas automatizadas aprobadas con SQLite, instalación desde cero y seeders repetidos. Frontend compilado y auditorías de Composer/npm sin vulnerabilidades reportadas. GitHub Actions incluye una matriz SQLite/MySQL; MySQL requiere ejecutar esa validación en un servidor o CI.

Antes de exponer

Recorrer login de ambos roles -> CRUD -> préstamo -> intento duplicado -> devolución -> historial. Probar solicitud, aprobación y entrega de reserva. Ejecutar **php artisan schedule:work** para procesar reservas vencidas. Las cuentas nuevas verifican su correo; en modo local el enlace queda en storage/logs/laravel.log.

Entrega mediante una rama y Pull Request hacia main. El equipo revisa los cambios antes de fusionarlos. El README contiene instalación, reglas de reservas y reparto de responsabilidades.